K8S-12: Specialized Monitoring Scenarios

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 12 of 13 | **Created:** January 2026 | **Last Updated:** 05/09/2026

NGINX Ingress, CSI Driver, Resource Tuning, and StatsD Ingestion

This notebook covers specialized monitoring scenarios including NGINX Ingress Controller instrumentation, CSI Driver architecture, resource sizing guidelines, and StatsD metric ingestion on Kubernetes.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
Kubernetes Cluster	Dynatrace Operator v1.0+ installed
NGINX Ingress	ingress-nginx controller (for Section 1)
Knowledge	Completed K8S-01 through K8S-02

1. NGINX Ingress Controller Monitoring

Why Monitor NGINX Ingress?

The NGINX Ingress Controller is often the entry point for all external traffic. Instrumenting it provides:

Benefit	Description
End-to-end traces	Complete request path visibility
Ingress latency	Measure time spent in ingress layer
Error correlation	Link 5xx errors to backend services
Traffic analysis	Understand traffic patterns

Prerequisites for NGINX Instrumentation

Requirement	Details
OneAgent version	1.227+
Pod naming	Must contain `ingress-nginx-`
Architecture	x86-64 only
Controller type	ingress-nginx (not nginx-ingress)

Configuration Steps

Step 1: Edit the ingress-nginx-controller ConfigMap

```
kubectl edit configmap ingress-nginx-controller -n ingress-nginx
```

Step 2: Add the OneAgent module loading snippet

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-
x86-64/liboneagentnginx.so;
```

Step 3 (Optional): Add trace context to access logs

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-
x86-64/liboneagentnginx.so;
  log-format-upstream: &gt;-
    $remote_addr - $remote_user [$time_local] "$request"
    [!dt
dt.trace_id=$dt_trace_id,dt.span_id=$dt_span_id,dt.trace_sampled=$dt_trace_sa
mpled]
    $status $body_bytes_sent "$http_referer" "$http_user_agent"
```

Step 4: Rolling restart the ingress controller

```
kubectl rollout restart deployment ingress-nginx-controller -n ingress-nginx

# Watch rollout
kubectl rollout status deployment ingress-nginx-controller -n ingress-nginx
```

Verifying NGINX Instrumentation

```
# Check OneAgent injection
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- cat
/proc/1/maps | grep dynatrace

# Check for errors in logs
kubectl -n ingress-nginx logs -l app.kubernetes.io/name=ingress-nginx | grep
-i dynatrace
```

> **Note:** No DynaKube changes are required for NGINX monitoring - only the ConfigMap edit and pod restart.

```
// Check if NGINX Ingress spans are arriving
fetch spans, from:-1h
| filter contains(service.name, "nginx") or contains(service.name, "ingress")
| summarize count = count(), by:{service.name, span.name}
| sort count desc
| limit 20
```

2. CSI Driver Architecture

Understanding the CSI Driver

The CSI (Container Storage Interface) Driver provides OneAgent code modules to application pods via volumes instead of host mounts.

CSI Driver Container Structure

The CSI Driver DaemonSet runs **5 containers** in a sidecar pattern:

Container	Purpose	Resource Impact
`csi-init`	Initialize volume plugin	Runs once at startup
`server`	Main CSI plugin	Core functionality
`provisioner`	Volume provisioning	Highest resource needs
`registrar`	Node registration	Low overhead
`livenessprobe`	Health checking	Minimal

3. CSI Driver Resource Configuration

Helm Values for CSI Driver

Configure resources for each CSI Driver container in your Helm `values.yaml`:

```yaml
csidriver:
  enabled: true

  # Init container - runs once at startup
  csiInit:
    resources:
      requests:
        cpu: 50m
        memory: 100Mi
      limits:
        cpu: 100m
        memory: 128Mi

  # Main CSI plugin
  server:
    resources:
      requests:
        cpu: 50m
        memory: 100Mi
      limits:
        cpu: 100m
        memory: 128Mi

  # Volume provisioner - needs more resources
  provisioner:
    resources:
      requests:
        cpu: 300m
        memory: 100Mi
      limits:
        cpu: 500m         # Often missing in defaults!
        memory: 256Mi     # Often missing in defaults!

  # Node registration sidecar
  registrar:
    resources:
      requests:
        cpu: 20m
        memory: 30Mi
      limits:
        cpu: 50m
        memory: 64Mi

  # Health check sidecar
  livenessprobe:
    resources:
      requests:
```

```
        cpu: 20m
        memory: 30Mi
      limits:
        cpu: 50m
        memory: 64Mi
```

CSI Driver Resource Summary Table

Container	CPU Request	CPU Limit	Memory Request	Memory Limit	Notes
`csiInit`	50m	100m	100Mi	128Mi	Init container
`server`	50m	100m	100Mi	128Mi	Main plugin
`provisioner`	300m	500m	100Mi	256Mi	**Add limits!**
`registrar`	20m	50m	30Mi	64Mi	Sidecar
`livenessprobe`	20m	50m	30Mi	64Mi	Sidecar

> **Warning:** Default `values.yaml` may be missing limits for the `provisioner` container. Add them to ensure predictable resource usage.

4. Component Sizing Guidelines

ActiveGate Sizing

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

OneAgent Resources

Mode	CPU Request	CPU Limit	Memory Request	Memory Limit
cloudNativeFullStack	100m	300m	256Mi	512Mi
classicFullStack (legacy)	150m	500m	512Mi	1Gi
applicationMonitoring	50m	200m	128Mi	256Mi

OTel Collector Sizing

Telemetry Volume	CPU Limit	Memory Limit	Use Case
Low	200m	256Mi	Dev/Test
Medium	500m	512Mi	Staging
High	1000m	1Gi	Production
Very High	2000m	2Gi	High-throughput

Log Monitoring Sizing

Log Volume	CPU Limit	Memory Limit	Notes
Low	200m	256Mi	< 1GB/day
Medium	500m	512Mi	1-10GB/day
High	1000m	1Gi	10-50GB/day
Very High	2000m	2Gi	50GB+/day

5. Telemetry Ingest Configuration

Multi-Protocol Ingestion

Configure DynaKube to accept telemetry from multiple sources:

```
spec:
  telemetryIngest:
    protocols:
      - otlp      # OpenTelemetry Protocol
      - jaeger    # Jaeger traces
      - zipkin    # Zipkin traces
      - statsd    # StatsD metrics
    serviceName: telemetry-ingest
```

Protocol Endpoints

Protocol	Default Port	Endpoint
OTLP gRPC	4317	`telemetry-ingest.dynatrace:4317`
OTLP HTTP	4318	`http://telemetry-ingest.dynatrace:4318`
Jaeger	14268	`http://telemetry-ingest.dynatrace:14268`
Zipkin	9411	`http://telemetry-ingest.dynatrace:9411`
StatsD	8125	`telemetry-ingest.dynatrace:8125` (UDP)

Application Configuration

```
# Application deployment using OTLP
env:
  - name: OTEL_EXPORTER_OTLP_ENDPOINT
    value: "http://telemetry-ingest.dynatrace:4318"
  - name: OTEL_SERVICE_NAME
    value: "my-application"
```

```
// Check CSI driver pod status
fetch logs, from:-1h
| filter matchesPhrase(content, "csi") and matchesPhrase(content,
"dynatrace")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Monitor telemetry ingest volume by protocol
fetch spans, from:-1h
| filter isNotNull(otel.scope.name)
| summarize span_count = count(), by:{service.name}
| sort span_count desc
| limit 15
```

6. StatsD Metrics Ingestion on Kubernetes

The Challenge

Many applications emit custom metrics using the StatsD protocol (UDP port 8125). On VMs, Dynatrace OneAgent includes a built-in StatsD daemon — but this feature is **not available** when OneAgent is deployed on Kubernetes via the Dynatrace Operator.

Ingestion Approaches for Kubernetes

Approach	Works on K8s?	Notes
OpenTelemetry Collector	**Yes**	Recommended. Officially supported and documented by Dynatrace.
OneAgent StatsD daemon	No	Only available on VM/host installs
ActiveGate remote StatsD	No	Containerized ActiveGate lacks the required extension module
Telegraf + Dynatrace plugin	Yes	Works but not officially documented by Dynatrace

Recommended: OpenTelemetry Collector with StatsD Receiver

Deploy a dedicated OTel Collector that listens for StatsD traffic, converts it to OTLP, and ships it to Dynatrace. The collector is **shared infrastructure** — it gets its own Deployment, separate from your application pods.

Deployment Pattern Options

Pattern	When to Use	Notes
Cluster-wide Deployment	Most common starting point	Single Deployment in a `monitoring` namespace. All apps send StatsD via FQDN.
Per-namespace Deployment	Isolated teams/environments	One collector per namespace. More isolation, more overhead.
DaemonSet	Very high metric volume	One pod per node. Avoids cross-node traffic. Overkill for most StatsD use cases.

> **Recommendation:** Start with a cluster-wide Deployment in a dedicated `monitoring` or `observability` namespace. The platform/infra team manages the collector independently from application deployments.

Step-by-Step Setup

Step 1: Create the collector configuration

```
# otelcol-config.yaml (stored as a ConfigMap)
receivers:
  statsd:
    endpoint: "0.0.0.0:8125"
    timer_histogram_mapping:
      - statsd_type: "histogram"
        observer_type: "histogram"
        histogram: {}
      - statsd_type: "timer"
        observer_type: "histogram"
        histogram: {}

processors:
  batch: {}

exporters:
  otlphttp:
    endpoint: "${DT_ENDPOINT}" # https://&lt;env-
id&gt;.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: "Api-Token ${DT_API_TOKEN}"

service:
  pipelines:
    metrics:
      receivers: [statsd]
      processors: [batch]
      exporters: [otlphttp]
```

Step 2: Deploy the collector in a dedicated namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-collector-statsd
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: otel-collector-statsd
  template:
    metadata:
      labels:
        app: otel-collector-statsd
    spec:
      containers:
        - name: otel-collector
          image: otel/opentelemetry-collector-contrib:0.96.0
          args: ["--config=/conf/otelcol-config.yaml"]
          ports:
            - containerPort: 8125
              protocol: UDP
          env:
            - name: DT_ENDPOINT
              valueFrom:
                secretKeyRef:
                  name: dynatrace-secret
                  key: endpoint
            - name: DT_API_TOKEN
              valueFrom:
                secretKeyRef:
                  name: dynatrace-secret
                  key: api-token
          volumeMounts:
            - name: config
              mountPath: /conf
      volumes:
        - name: config
          configMap:
            name: otel-collector-config
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: otel-collector-statsd
  namespace: monitoring
spec:
  selector:
    app: otel-collector-statsd
  ports:
    - name: statsd-udp
      protocol: UDP
      port: 8125
      targetPort: 8125
```

Step 3: Point StatsD clients at the collector using the FQDN

Since the collector lives in the `monitoring` namespace, apps in other namespaces must use the fully qualified service name:

```
# In your application deployment (any namespace)
env:
  - name: STATSD_HOST
    value: "otel-collector-statsd.monitoring.svc.cluster.local"
  - name: STATSD_PORT
    value: "8125"
```

> **Note:** No application code changes are needed — just the env var configuration. The collector is managed independently from your application lifecycle.

Ownership Model

Component	Owner	Namespace
OTel Collector Deployment	Platform / Infra team	`monitoring`
Collector ConfigMap + Secret	Platform / Infra team	`monitoring`
Application `STATSD_HOST` env var	App team	App namespace

Requirements

Requirement	Details
API Token	`metrics.ingest` scope
Environment URL	`https://<env-id>.live.dynatrace.com`
Collector Image	`otel/opentelemetry-collector-contrib` (includes StatsD receiver)

> **Tip:** You can also use the Dynatrace Collector image or build a custom collector with the OpenTelemetry Builder. The key requirement is the StatsD receiver component.

Verifying StatsD Metrics in Dynatrace

After deploying the collector and pointing StatsD clients at it, verify metrics are flowing:

```
// Verify StatsD metrics are being ingested via OTel Collector
// StatsD metrics arrive as OTLP metrics — query them by metric key prefix
timeseries values = avg(statsd.my_app.request_count), from:-1h
| fieldsAdd avgValue = arrayAvg(values)
| sort avgValue desc
| limit 10
```

7. Troubleshooting Specialized Scenarios

NGINX Ingress Issues

Issue	Cause	Solution
Module load fails	Wrong architecture	x86-64 only, no ARM
No spans from ingress	Pod name mismatch	Must contain `ingress-nginx-`
Partial traces	OneAgent version	Upgrade to 1.227+

```
# Debug NGINX module loading
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- \
  ls -la /opt/dynatrace/oneagent-paas/agent/bin/current/
```

CSI Driver Issues

Issue	Cause	Solution
Volume mount fails	Provisioner OOM	Increase provisioner limits
Slow pod startup	CSI driver overloaded	Add resources, check node count
Code modules missing	CSI not enabled	Enable in Helm values

```
# Check CSI driver status
kubectl -n dynatrace get pods -l app.kubernetes.io/component=csi-driver

# View CSI driver logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=csi-driver -c server
```

StatsD Ingestion Issues

Issue	Cause	Solution
No metrics arriving	Wrong collector image	Use `otel/opentelemetry-collector-contrib` (not base)
UDP traffic not reaching collector	Missing Service or wrong protocol	Ensure Service specifies `protocol: UDP`
Metrics visible but unnamed	Missing metric prefix	Configure StatsD client to include application prefix
Token errors in collector logs	Wrong scope	Token needs `metrics.ingest` scope

```
# Check OTel collector logs for StatsD errors
kubectl logs -l app=otel-collector-statsd --tail=50

# Test UDP connectivity to the collector
kubectl run statsd-test --rm -it --image=busybox -- \
  sh -c 'echo "test.metric:1|c" | nc -u -w1 otel-collector-statsd 8125'
```

Resource Exhaustion

Symptom	Component	Action
OOMKilled	Any	Increase memory limits
CPU throttling	Any	Increase CPU limits
Slow queries	ActiveGate	Add replicas or resources
Dropped spans	OTel Collector	Increase batch size, resources

Summary

In this notebook, you learned:

- **NGINX Ingress monitoring** with OneAgent module loading
- **CSI Driver architecture** and the 5-container structure
- **CSI Driver resource configuration** with per-container limits
- **Component sizing guidelines** for all Dynatrace components
- **Telemetry ingest configuration** for multi-protocol support
- **StatsD ingestion on Kubernetes** via the OpenTelemetry Collector (the only supported approach for K8s)
- **Troubleshooting** common specialized monitoring issues

References

- NGINX instrumentation (DT docs)
- Storage / CSI driver (DT docs)
- Dynatrace Operator Helm chart (Dynatrace GitHub)
- Telemetry ingest with OpenTelemetry (DT docs)
- StatsD ingestion (DT docs)
- StatsD via OpenTelemetry Collector (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
